

ITU-MiniTwit - Group e - Report

Frederik Hørup , Marie Johansen , Nikolej Lundquist , Sara Bagger , Vitus Brodersen <>

May 16, 2026

Contents

1	Introduction	1
2	System	1
2.1	Architecture and Design	1
2.2	Dependencies	1
2.3	System States	1
3	Process	2
3.1	CI/CD	2
3.2	Logging	2
3.3	Security	3
3.4	Availability and Scaling	4
4	Reflection	4
5	Use of Generative AI	4
6	References	4

1 Introduction

Authors:

2 System

2.1 Architecture and Design

Authors:

2.2 Dependencies

Authors:

2.3 System States

Authors:

3 Process

3.1 CI/CD

Authors: Vitus

The CI/CD process is based on GitHub Actions workflows that run on the `main` branch or on a scheduled timer. The MiniTwit repository currently uses the following active workflows to do CI/CD:

- **Static Analysis** — runs on push and pull request events for `main`. It performs:
 - Dockerfile linting with Hadolint against `Dockerfile-MiniTwit`.
 - .NET setup and tool restore.
 - C# formatting validation with `dotnet csharpier check ..`
 - CodeQL initialization for C#.
 - Dependency restore and a strict Roslyn build with `TreatWarningsAsErrors=true`.
 - CodeQL analysis to surface static security issues.
 - Docker Scout scanning for critical vulnerabilities on the DockerHub image.
 - **Build and test** — also runs on push and pull request events for `main`. It restores dependencies, performs a clean build, and runs the test suite.
 - **SonarCloud & Codacy** — provide external static analysis, code smell detection, and quality gating on pushes as well.
 - **Continuous Deployment (Deploy To DO)** — runs on pushes to `main` and can also be triggered manually. It builds and pushes the Docker images for and deploys them to the Digital Ocean Droplet.
 - **automatic-weekly-release** — runs on schedule every Tuesday at 08:00 UTC and can be started manually. It builds the project, runs tests, packages release artifacts, and creates a GitHub release.
- ## Monitoring

Authors: Marie

The systems monitoring is setup using the open-source monitoring system Prometheus in collaboration with Grafana for visualizing and querying the metrics. (TODO: references for grafana and prometheus) The `app.MapMetrics()`; and `app.UseHttpMetrics()`; middleware were added to the pipeline. `app.MapMetrics()`; exposes the HTTP endpoint for Prometheus to scrape and `app.UseHttpMetrics()`; collect Prometheus metrics for processed HTTP requests (from documentation of `UseHttpMetrics`).

The monitoring is pull based as the application exposes metrics which are then pulled by Prometheus.

In grafana, the monitoring has been split up into two dashboards; application metrics and infrastructure metrics (TODO: Insert reference to video). Application metrics focuses mostly on request rates and displays: CPU usage in seconds and the amount of HTTP request received as well as split into different types of requests. Infrastructure metrics focuses on the server side and displays dashboards containing information about: memory usage, CPU usage and process uptime.

The applications monitoring is at the reactive level as a small amount of dashboard that are mostly operationally-focused are provided and the broad focus is on measuring availability. However, the monitoring has not moved towards monitoring data to measure user experience or that the business side would benefit from. (TODO: find reference - his reference on slides does not work)

There are many ways monitoring could have been improved. For one, database monitoring would have been especially beneficial both for the operational side and to provide metrics for the business side e.g. number of users in the system.

Lastly, the monitoring dashboards provided by Digital Ocean to monitor the VMs has been regularly used.

3.2 Logging

Authors: Nikolej

Todo: - [x] Architectural intent - [] Operational usefulness - [] Awareness of limitations

The system uses a minimal push-based logging stack utilizing Alloy, Loki and Grafana, focusing on aggregation and visualization rather than analysis. The Grafana-Loki stack was primarily chosen because it is cheap to run and for its native support for Grafana, which the existing monitoring system was already using.

Alloy collects logs from running Docker containers through the Docker socket `host = unix:///var/run/docker.sock`, where they are populated with metadata like labels before being forwarded to Loki for storage. Lastly, the logs are queried by Grafana and presented visually.

As mentioned, the priority of this setup is aggregation and visualization, which is accomplished by grouping logs by containers in Grafana. The default Grafana Logs Drilldown page is more than sufficient for this purpose, hence there are no custom dashboards.

This setup improves observability significantly, allowing easier searching and faster response in case of errors.

TODO: Hypothetical operational usefulness

TODO: Something about which categories of logs we have: - Minitwit - DB queries - HTTP requests - Errors

3.3 Security

Authors: Sara

We applied several security hardening measures to our system across infrastructure, network, CI/CD, and container configuration.

Reverse proxy/TLS. First, we deployed a Nginx reverse proxy to the application and enabled HTTPS using TLS certificates. The reverse proxy terminates incoming HTTPS traffic and forwards requests internally to the application containers. This improves security by encrypting communication between clients and the server and by reducing direct exposure of the application itself. We used Let's Encrypt certificates together with automatic renewal mechanisms to avoid manual certificate management.

Firewall. To protect the servers themselves, we configured a UFW software firewall. We followed the principle of least privilege by allowing only required traffic such as SSH and HTTP/HTTPS while denying unnecessary incoming connections. This was important to avoid unintentionally exposing services externally if firewall rules are misconfigured. We also considered firewall logging to detect suspicious or blocked traffic patterns, but because of lack of time, this was omitted.

CI/CD. In the CI/CD pipeline, we integrated automated security analysis tools to support a shift-left security approach, where vulnerabilities are detected before deployment. We added GitHub CodeQL analysis to statically scan the application source code for known security vulnerabilities and insecure coding patterns. Additionally, we integrated Docker Scout to scan container images for known common vulnerabilities and exposures and vulnerable dependencies. The pipeline was configured to fail on critical vulnerabilities, preventing insecure images from being deployed automatically.

Docker hardened images. We also hardened our Docker environment by switching several production services to Docker Hardened Images (DHI). Specifically, we replaced standard Grafana, Prometheus, and ASP.NET images with hardened variants from dhi.io. These images are designed to reduce attack surfaces by minimizing unnecessary packages and dependencies.

Container execution. Beyond changing base images, we further hardened container execution. For example, Grafana was configured to run as a non-root user instead of running with root privileges.

We also introduced initialization steps to ensure correct permissions on mounted volumes without granting unnecessary privileges to the running application containers.

A key lesson from the course was that security should not rely on a single mechanism. Therefore, our approach combined multiple layers of protection: encrypted communication through TLS, restricted network access through firewalls, automated vulnerability scanning in CI/CD, hardened container images, and safer runtime configurations. This follows a defense-in-depth strategy, where multiple independent security mechanisms reduce the likelihood that a single vulnerability compromises the entire system.

3.4 Availability and Scaling

Authors: Frederik

To increase availability of the system a simple load balancing setup has been made. This can also be seen in the image from architecture and design. Here a load balancer balances the load of each server running the application. To help further for availability a second backup load balancer exists to take over if the primary fails. For scaling the application has not been scaled further than being on 2 servers. It should have been so each server had at least 2 running instances of the application and a corresponding update strategy should have been implemented in the CI/CD pipeline. If it had been implemented the blue-green upgrade strategy would have been implemented.

4 Reflection

Authors:

5 Use of Generative AI

Authors: Nikolej

During the development of this project we used ChatGPT in two main ways:

Debugging. Often when encountering unexpected bugs and error messages, we would consult ChatGPT about the cause and potential fixes. While it could rarely fix the issues entirely by itself, more often than not, it pointed us in the right direction.

Research. This course presents challenges involving numerous technologies, most of which we were unfamiliar with. As such, ChatGPT was used to summarize lengthy documentation and provide concrete guides tailored to our situation.

Generating boilerplate / configurations. ChatGPT was also used for simple tasks like generating boilerplate code or writing Github Actions Workflow files. For example, the `build-report.yml` workflow, that converts the markdown source into a pdf.

The use of AI has made these tasks easier, spared us many frustrations and saved considerable time during development. On the flip side, AI may have deprived us the extensive knowledge and experience you gain by, for example, painstaking reading of documentation or spending hours resolving a tiny bug.

6 References